RestauSpace

System design, CI/CD et exploitation

Document	Référence technique interne (exploitation)
Produit	SaaS multi-tenant de gestion de restaurants : console, site public, back-office, API
Éditeur	HBA Tech et Trade
Version	25 octobre 2026
Diffusion	Interne. Contient des noms de serveurs, adresses et chemins : ne pas publier.

Sommaire

1. Vue d'ensemble et infrastructure
2. Architecture logicielle
3. Données et base PostgreSQL
4. Sécurité
5. Applications web (fronts)
6. Intégrations externes
7. CI/CD Jenkins
8. Déploiement
9. Sauvegardes et restauration
10. Observabilité
11. Exploitation : commandes, incidents, points ouverts

Sources : dépôt `Hectorberi01/RestauSpace` (README, `deploy/`, `Jenkinsfile`, `docs/`) à la date du document.

1. Vue d'ensemble et infrastructure

RestauSpace est un **monolithe modulaire .NET 10** (une seule API, 26 modules isolés) servi derrière Traefik avec trois applications Next.js. La base PostgreSQL est sur un serveur séparé, joint par le réseau privé ; l'observabilité est sur un troisième serveur.

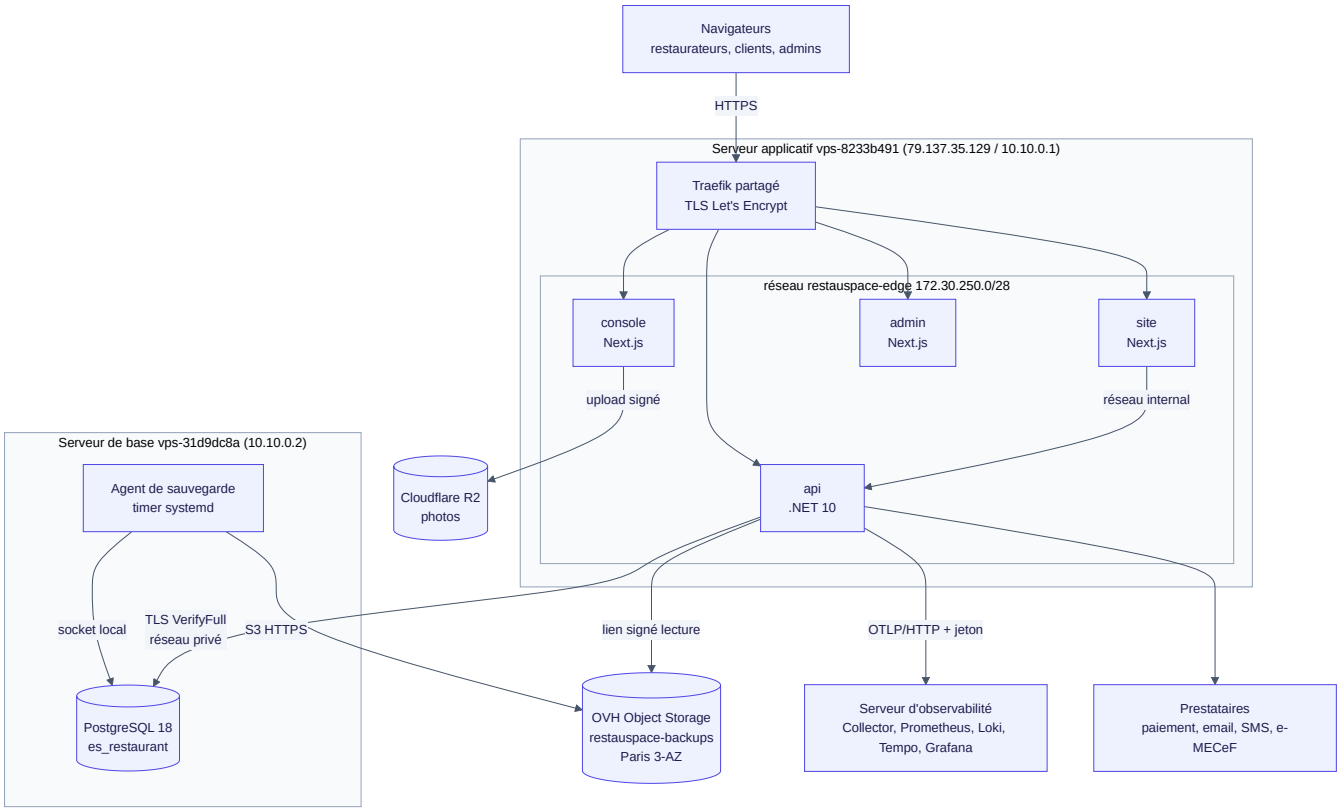


Figure 1 : topologie de production

Serveurs

Rôle	Hôte	Adresses	Contenu
Applicatif	`vps-8233b491`	public `79.137.35.129`, privé `10.10.0.1`	Traefik (partagé avec un autre projet), conteneurs `api`, `console`, `site`, `admin`, dépôt `/opt/restauspace`
Base de données	`vps-31d9dc8a`	privé `10.10.0.2`	PostgreSQL 18 natif (paquet Debian), agent de sauvegarde (systemd)
Observabilité	serveur dédié	`OBSERVABILITY_OTLP_URL`	Collector OpenTelemetry, Prometheus, Loki, Tempo, Grafana, Alertmanager
CI/CD	Mac (Jenkins dans Docker Desktop)	non exposé	Jenkins 2.580, Testcontainers, build des images

Noms de domaine

Nom	Service	Accès
`restauspace.com`, `www`	site public (vitrine, commande en ligne, réservation)	public ; www redirigé vers la racine
`app.restauspace.com`	console des restaurateurs (dont la caisse tactile)	public, authentifié
`api.restauspace.com`	API .NET	public ; `/v1/admin` limité par `ADMIN_ALLOWED_IPS`
`admin.restauspace.com`	back-office de la plateforme	`ADMIN_ALLOWED_IPS` + 2FA obligatoire

Réseaux Docker du serveur applicatif

Réseau	Membres	Rôle
`restauspace-edge` (172.30.250.0/28, externe)	Traefik + les 4 services	seul point d'entrée HTTP ; seule source acceptée pour `X-Forwarded-For` (`EDGE_SUBNET`)
`internal`	`site`, `api`	rendu serveur du site : `API_INTERNAL_URL=http://api:8080`
`default`	`migrations`, `db-grants`	sortie vers la base pendant le déploiement

Aucun conteneur RestauSpace n'est sur le réseau `proxy` de l'autre projet ; aucun port n'est publié sur l'hôte. `deploy.sh` vérifie le sous-réseau et signale tout conteneur étranger sur `restauspace-edge`.

2. Architecture logicielle

Un seul exécutable, `SaasRestaurant.Host`, compose les modules. Chaque module a quatre projets (Domain, Application, Infrastructure, Api) et son propre schéma PostgreSQL. Les modules pourront être extraits en services sans réécriture du domaine.

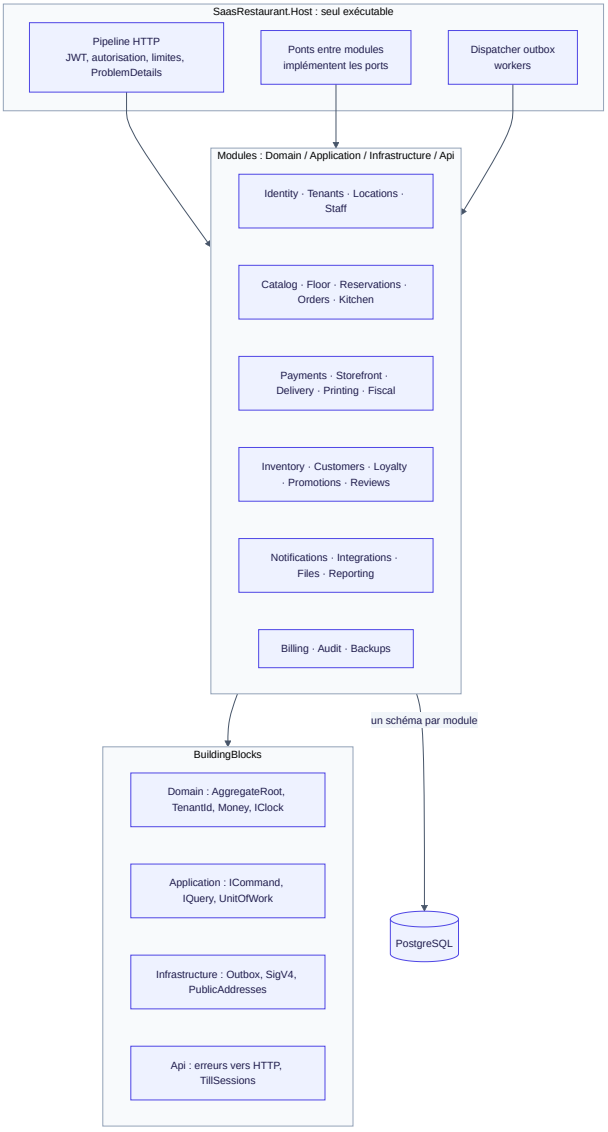


Figure 2 : découpage en modules et couches

Règles d'architecture (vérifiées par `Tests/Architecture`)

Règle	Contrôle
Domain ne référence ni EF Core, ni ASP.NET Core, ni Npgsql, ni les autres couches	`LayerDependencyTests`
Application ne référence ni EF Core, ni ASP.NET Core, ni Infrastructure/Api	`LayerDependencyTests`
Aucun module ne référence l'assembly d'un autre module	`LayerDependencyTests`
Tout agrégat porte son `TenantId` (sauf objets plateforme : `User`, `Tenant`, `Plan`…)	`DomainModelTests`
Pas de setter public, collections non modifiables, événements immuables	`DomainModelTests`
Un module ne lit jamais le schéma d'un autre	un schéma et un DbContext par module
Une table d'un tenant ne montre que le tenant courant	filtre EF global (`TenantDbContext`)

Communication entre modules

- **Synchrone** : le module demandeur déclare un **port** ; le Host l'implémente par un pont (`*Bridges.cs`) qui appelle l'Application de l'autre module. Les données entrent comme références ou instantanés, jamais comme objets d'un autre module.
- **Asynchrone** : événements de domaine → **outbox** dans la même transaction → dispatcher → consommateurs idempotents (audit, notifications, webhooks sortants, projections de reporting).

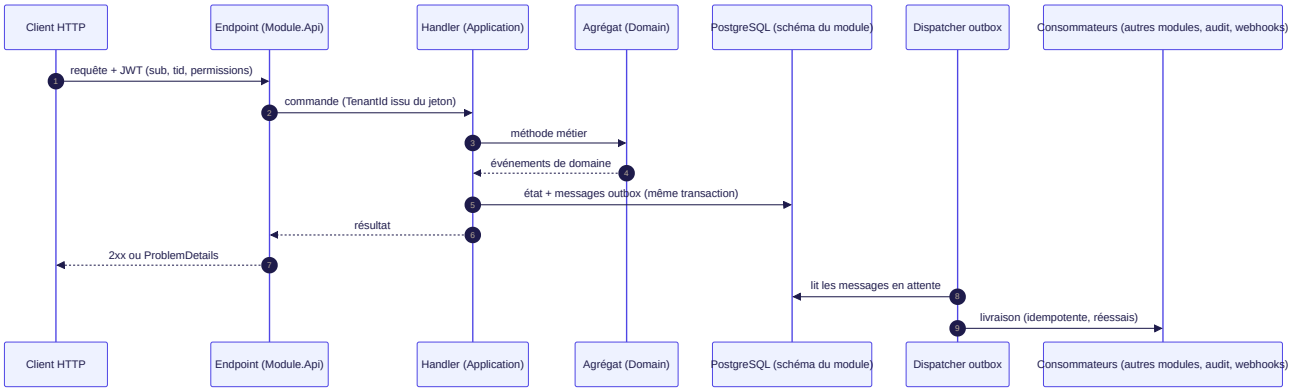


Figure 3 : traitement d'une commande HTTP et publication des événements

Modules

Domaine	Modules et responsabilité
Accès et organisation	**Identity** (comptes, sessions, 2FA, invitations) · **Tenants** (restaurant client, devise, fuseau) · **Locations** (établissements, horaires) · **Staff** (employés, rôles, permissions, caisses)
Salle et commande	**Catalog** (produits, menus, prix) · **Floor** (salles, tables, QR) · **Reservations** · **Orders** (agrégat central) · **Kitchen** (tickets, postes)
Vente et encaissement	**Payments** (paiement client, remboursements, prestataires) · **Storefront** (vente en ligne, clés de site) · **Delivery** · **Printing** (tickets) · **Fiscal** (facture normalisée e-MECeF, Bénin)
Relation client	**Customers** (CRM, consentements) · **Loyalty** · **Promotions** (codes promo) · **Reviews**
Plateforme	**Inventory** (stocks, recettes) · **Notifications** (email, SMS, WhatsApp, push) · **Integrations** (webhooks sortants, clés d'API) · **Files** (médias, R2) · **Reporting**
Administration SaaS	**Billing** (plans, abonnements, quotas, options) · **Audit** (journal append-only) · **Backups** (sauvegardes, restaurations)

3. Données et base PostgreSQL

Organisation

- Une base `es_restaurant`, **un schéma par module** (`identity`, `orders`, `billing`, `backups`…), chacun avec sa table d'outbox et son historique de migrations EF.
- Migrations appliquées **uniquement au déploiement** par le conteneur `migrations` (`--migrate-only`) ; `Database__MigrateOnStartup=false` en production.
- Concurrence optimiste par `xmin` ; contraintes d'unicité partielles pour les invariants (ex. une seule sauvegarde active).

Comptes

Compte	Droits	Utilisé par
`postgres`	superutilisateur ; connexion locale uniquement (peer)	administration, agent de sauvegarde
`es_restaurant_user`	propriétaire des schémas, NOSUPERUSER	conteneurs `migrations` et `db-grants`
`es_restaurant_app`	SELECT/INSERT/UPDATE/DELETE uniquement (posés par `db-grants.sql` à chaque déploiement)	conteneur `api`

Accès réseau

- PostgreSQL écoute sur l'adresse privée ; `pg_hba.conf` n'accepte que `hostssl` depuis `10.10.0.1/32` pour la base et ses deux comptes, et refuse tout le reste.
- **TLS vérifié** : autorité propre au projet (`deploy/database/tls-ca.sh`), certificat serveur avec SAN `IP:10.10.0.2`, clients en `VerifyFull` (Npgsql) et `verify-full` (psql). Le certificat de l'autorité est monté dans `api`, `migrations` et `db-grants` depuis `deploy/production/tls/db-ca.crt`.
- `deploy.sh` refuse de démarrer en `VerifyFull` sans ce fichier et avertit tant que le mode reste `Require`.

Fichiers et médias

Les photos sont stockées dans Cloudflare R2 : la console envoie et lit directement par URL signée (module Files, signature SigV4). Les sauvegardes sont dans OVH Object Storage (section 9).

4. Sécurité

Identité et sessions

- Jeton d'accès JWT court ; refresh token par **famille** (`{premier}.{secret}`), rotation à chaque usage, rejeu détecté → famille révoquée, durée de vie maximale `Identity:Sessions:FamilyMaxDays=90`.
- **TenantId** : uniquement le claim `tid` du jeton, posé après contrôle d'appartenance (`POST /me/tenant`). Jamais lu dans la requête.
- **Permissions** : rôle Staff revérifié à chaque requête (`RequirePermission`) ; un employé ne peut ni s'attribuer un rôle qu'il n'a pas, ni changer son propre rôle ; suspendre un propriétaire est réservé aux propriétaires.
- **Caisse tactile** : session dédiée de 12 h sans renouvellement (claims `till`, `till_loc`), liste blanche de permissions.
- **Administrateurs plateforme** : email vérifié et déclaré, jeton sans tenant ni caisse, **2FA obligatoire** ; tentatives 2FA limitées.

Secrets

- Clés applicatives (signature JWT, 2FA, chiffrement des identifiants prestataires, OTP, webhooks, fiscal, suivi, médias) générées par `generate-secrets.sh`, toutes différentes (contrôlé par `deploy.sh`), dans `api.env` (droits 600, hors dépôt).
- Identifiants des prestataires saisis par les restaurateurs : chiffrés en base, jamais renvoyés en clair, jamais dans les logs, événements, traces ou réponses.
- Aucune donnée personnelle dans les logs, événements, traces OpenTelemetry ni ticket imprimé.

Réseau et exposition

- Seul Traefik est exposé ; X-Forwarded-For accepté seulement depuis `EDGE_SUBNET`.
- Back-office et `/v1/admin` : middleware Traefik `rs-admin-allowlist` (`ADMIN_ALLOWED_IPS`).
- Site : CSP avec nonce sur les pages sensibles (commande, paiement, compte, borne…), COOP partout, URL de paiement filtrée (`safeCheckoutUrl`).
- Envois sortants (webhooks, SMTP) : adresses privées refusées (`PublicAddresses`), ports SMTP 25/465/587/2525 uniquement.
- Vente en ligne : 3 paiements en attente maximum par client, attente de 10 min, message d'erreur de coupon uniforme.

Traefik partagé

`exposedByDefault: false` côté Traefik ; tous les routeurs RestauSpace sont préfixés `rs-` ; socket Docker monté en lecture seule ; tableau de bord Traefik fermé. Après tout changement, vérifier qu'aucun autre conteneur ne déclare un routeur sur `*.restauspace.com`.

5. Applications web

Application	Dossier	Domaine	Rôle
Console	`restaurantos-app`	`app.`	gestion du restaurant : carte, salle, commandes, cuisine, caisse tactile (`/tactile`), paiements, notifications, intégrations, abonnement
Site	`restaurantos-site`	racine	vitrine, tarifs, documentation développeurs, commande en ligne, réservation, borne, suivi de commande
Back-office	`restaurantos-super-admin`	`admin.`	restaurants, plans, prestataires plateforme, notifications, journaux d'audit, **sauvegardes**

- Next.js 16 en mode *standalone*, image `node:22-alpine`, pnpm, utilisateur non root, port 3000.
- Les variables `NEXT_PUBLIC_*` sont **figées au build** (fichier `build.env` de Jenkins) : tout changement de domaine, contacts ou `STOREFRONT_KEY` demande une nouvelle livraison.
- Le site appelle l'API par le réseau Docker pour le rendu serveur, et par `https://api.restauspace.com` depuis le navigateur.

6. Intégrations externes

Domaine	Prestataires	Points d'entrée
Paiement des clients	FedaPay, KKiaPay, Stripe, PayPal (identifiants par restaurant)	webhooks signés `/v1/payments/webhooks/{code}/{configuration}`
Abonnement SaaS	prestataires de la plateforme (back-office)	`/payments/webhooks/{provider}`
Email	SMTP, Brevo, Mailjet, Resend, Amazon SES	sortant ; test de configuration binaire (OK ou échec)
Facture normalisée	e-MECeF (DGI Bénin)	module Fiscal
Développeurs	clés d'API (option payante) et webhooks sortants signés (Standard Webhooks)	module Integrations, doc sur le site
Stockage	Cloudflare R2 (médias), OVH Object Storage (sauvegardes)	signature SigV4

7. CI/CD Jenkins

Jenkins tourne sur le Mac (Docker Desktop, nécessaire aussi pour Testcontainers). Un seul `Jenkinsfile` est utilisé par **deux jobs** séparés, pour qu'une branche ou une demande de fusion ne puisse jamais lire les identifiants de livraison.

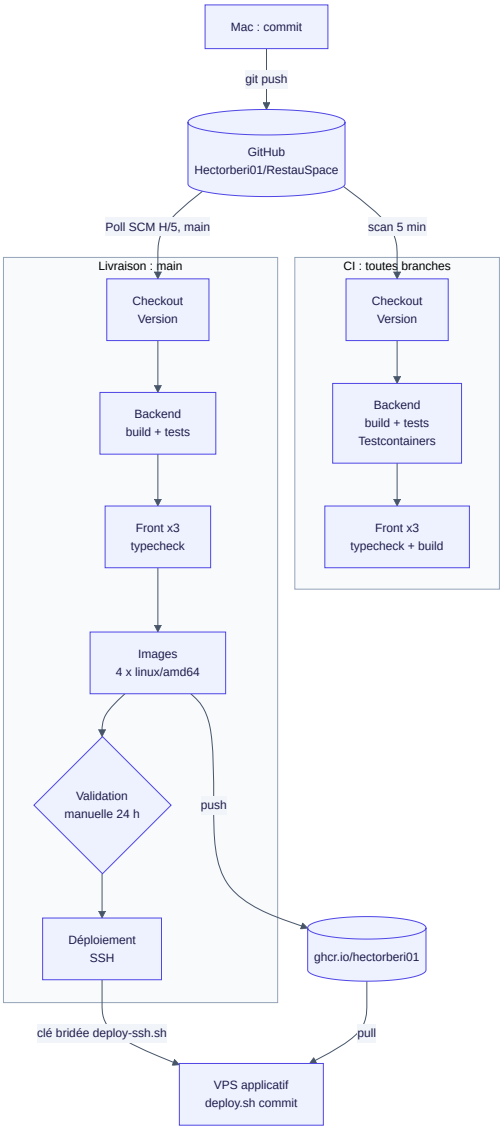


Figure 4 : intégration continue et livraison

Jobs

Job	Type	Déclencheur	Fait	Identifiants
`RestauSpace`	Multibranch	scan périodique, 5 min	tests .NET, typecheck et build des 3 fronts, sur toutes les branches	aucun
`release/restauspace-release`	Pipeline (SCM, `*/main`)	Poll SCM `H/5 * * * *`	mêmes vérifications, puis images, validation manuelle, déploiement	ceux du dossier `release`

Le Jenkinsfile distingue la livraison par `isRelease()` : pas de `BRANCH_NAME` (job Pipeline) et `GIT_BRANCH` = `main`. Dans le job Multibranch, les étapes Images, Validation et Déploiement sont donc sautées.

Étapes

Étape	Détail
Version	`RELEASE` = commit sur 12 caractères (même étiquette que les images et `deploy.sh`)
Backend	`dotnet restore`, `build -c Release`, `dotnet test` (résultats TRX archivés) ; tests d'architecture, tests Host sur un vrai PostgreSQL (Testcontainers)
Node.js	Node 22.20.0 installé une fois dans `JENKINS_HOME/tools`, pnpm à la version du `packageManager`
Front (parallèle)	console, site, back-office : `pnpm install --frozen-lockfile`, `pnpm typecheck`, `pnpm build` hors livraison
Images	`docker compose -f docker-compose.build.yml build --pull` puis `push` : 4 images `linux/amd64` vers `ghcr.io/hectorberi01/`
Validation	`input` « Déployer <commit> en production ? », délai 24 h
Déploiement	`ssh … "cd … && ./deploy.sh <commit>"` avec clé du VPS épinglée si `RESTAUSPACE_DEPLOY_KNOWN_HOSTS` est défini

Identifiants et variables

Nom	Type	Contenu
`ghcr`	Username/password	compte GitHub + jeton `write:packages`
`restauspace-build-env`	Secret file	`build.env` : `DOMAIN`, `STOREFRONT_KEY`, `FILES_ORIGINS`, contacts, réseaux sociaux
`restauspace-vps-ssh`	SSH private key	clé du compte de déploiement du VPS
`RESTAUSPACE_DEPLOY_HOST`	variable globale	`deploy@79.137.35.129` (ou le compte utilisé)
`RESTAUSPACE_DEPLOY_DIR`	variable globale	`/opt/restauspace/deploy/production`
`RESTAUSPACE_DEPLOY_KNOWN_HOSTS`	variable globale (facultative)	ligne `ssh-keyscan -t ed25519` vérifiée

Clé SSH bridée. Dans `authorized_keys` du VPS : `restrict,command="/opt/restauspace/deploy/production/deploy-ssh.sh" ssh-ed25519 …`. `deploy-ssh.sh` n'accepte que `./deploy.sh <commit hexadécimal>`. À n'activer qu'une fois ce script présent dans le dépôt du VPS.

Hygiène. `main` est livrée telle quelle : protéger la branche sur GitHub. Ne jamais commiter de dossier de travail (ex. `Claude outputs/`, ignoré par `.gitignore`), ni `.env`, `api.env`, `build.env`, `tls/`.

8. Déploiement

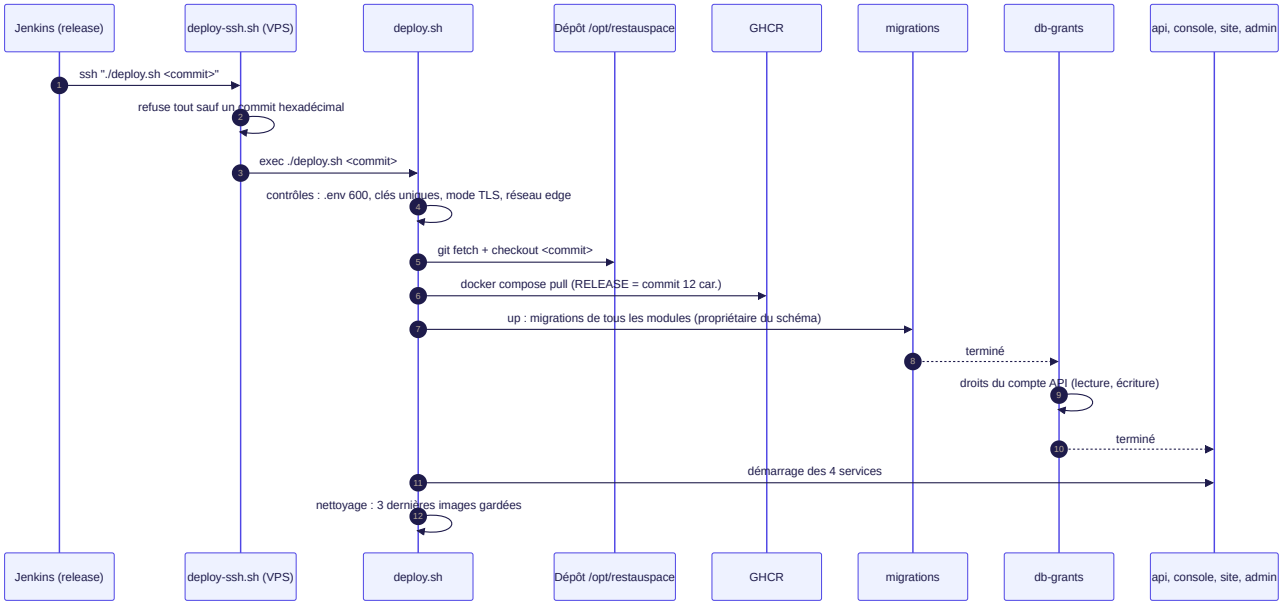


Figure 5 : séquence d'un déploiement

Services du compose de production (`deploy/production/docker-compose.yml`)

Service	Image	Rôle	Démarrage
`migrations`	`restauspace-api`	`--migrate-only` avec le propriétaire du schéma	s'exécute puis s'arrête
`db-grants`	`postgres:16-alpine`	`db-grants.sql` : droits du compte API sur tous les schémas	après `migrations`
`api`	`restauspace-api`	API, compte `es_restaurant_app`, pool 50, clés DataProtection en volume	après `db-grants`
`console`, `site`, `admin`	images Next.js	applications web	immédiat

Fichiers du serveur applicatif

Fichier	Contenu
`.env` (600)	base (`DATABASE_*`, `POSTGRES_*`), `DOMAIN`, `EDGE_SUBNET`, `IMAGE_REGISTRY`, `RELEASE`, `ADMIN_ALLOWED_IPS`, observabilité
`api.env` (600)	clés générées, SMTP, R2, administrateurs, `Backups__*` (clé OVH lecture seule)
`tls/db-ca.crt`	certificat public de l'autorité PostgreSQL

Contrôles de `deploy.sh` avant tout changement

1. `.env` et `api.env` présents, droits ramenés à 600.
2. Aucune clé de chiffrement ou de signature partagée entre deux usages.
3. Mode TLS de la base : `tls/db-ca.crt` exigé en `VerifyFull`.
4. Réseau `restauspace-edge` présent, sous-réseau égal à `EDGE_SUBNET`, conteneurs étrangers signalés.

Retour arrière

`./deploy.sh <commit précédent>`, à condition que les migrations soient compatibles (une migration n'est jamais annulée automatiquement). Les 3 dernières images de chaque application sont conservées sur le VPS.

> Les migrations sont générées par le développeur (`scripts/migrations-*.sh`) et commitées **avant** la livraison. Une migration manquante fait échouer le service `migrations`, donc le démarrage de l'API.

9. Sauvegardes et restauration

Le pg_dump ne tourne **jamais dans l'API**. L'API (module Backups) enregistre des demandes dans le schéma `backups` ; un agent sur le serveur de base les exécute en local, sans réseau entrant ni jeton.

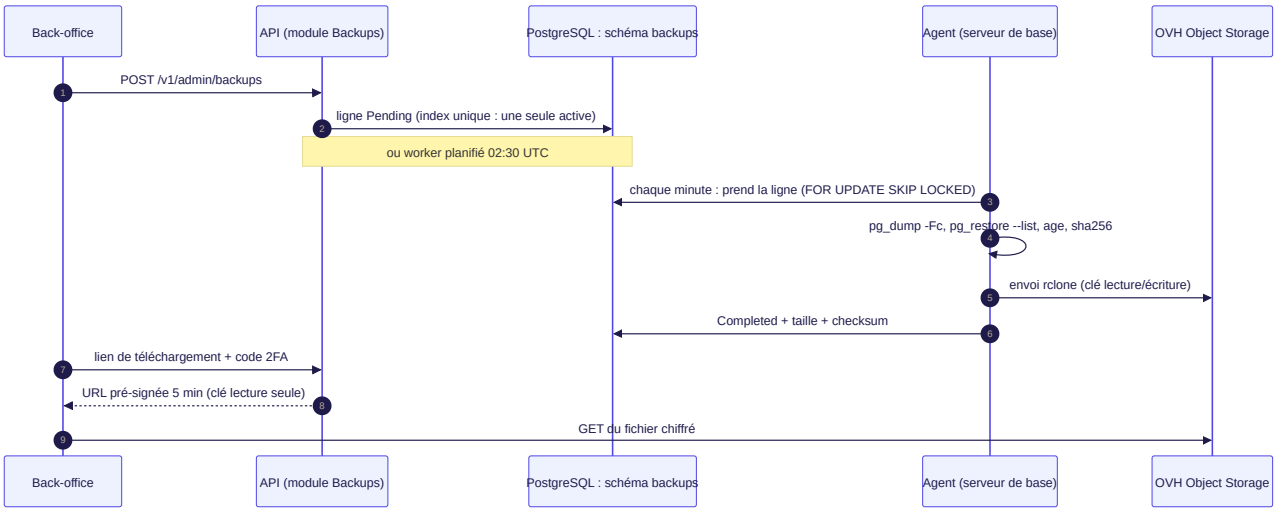


Figure 6 : sauvegarde et téléchargement

Composants

Élément	Où	Détail
Module Backups	API	routes `/v1/admin/backups*`, worker planifié (02:30 UTC), maintenance : sessions bloquées → `timeout`, expiration après 30 jours en gardant au moins 7
Agent	`/usr/local/sbin/restauspace-backup-agent`	timer systemd chaque minute, `flock`, configuration `/etc/restauspace-backup.env` (600)
Chiffrement	age	clé publique dans la configuration ; clé privée `/root/restauspace-backup/age-identity.txt` + copie hors ligne
Stockage	OVH, `restauspace-backups`, `eu-west-par`	clés `{base}/{aaaa}/{mm}/{id}.dump.age` ; chiffrement SSE-OMK en plus d'age
Utilisateurs S3	OVH	`restauspace-backup-agent` : liste, lecture, écriture, suppression ; `restauspace-backup-api` : liste et lecture (politiques `deploy/database/ovh-politique-*.json`)

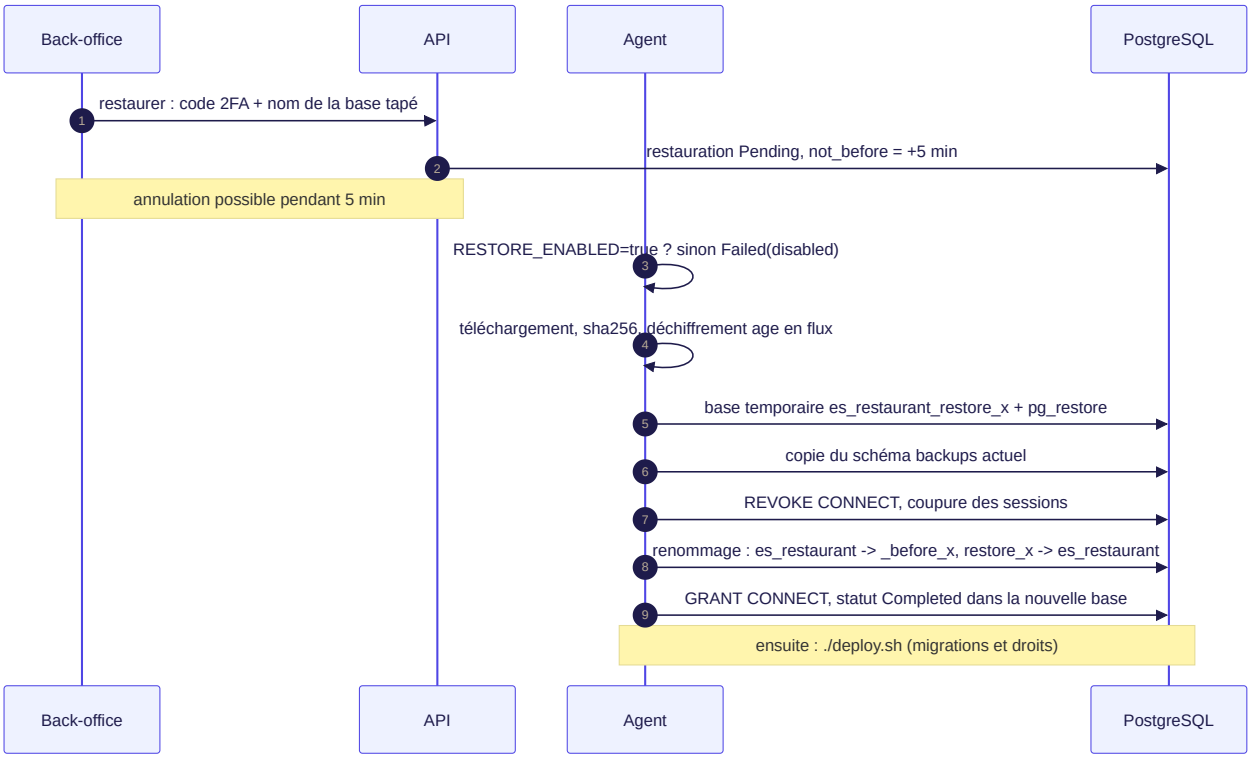


Figure 7 : restauration depuis le back-office

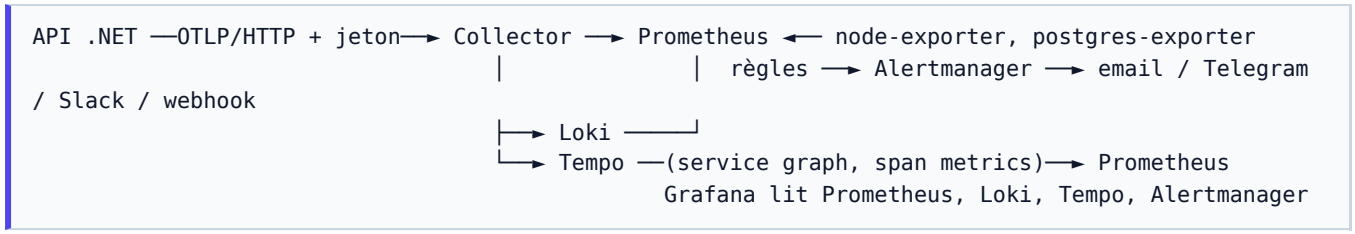
Garde-fous de la restauration

- Code 2FA + nom de la base tapé ; délai de 5 minutes annulable.
- Désactivée par défaut sur le serveur (`RESTORE_ENABLED=false`) : une API compromise ne peut pas en lancer.
- L'agent ne fait pas confiance aux lignes : base configurée uniquement, clé de stockage validée par expression régulière, sha256 vérifié avant déchiffrement.
- Ancienne base conservée (`es_restaurant_before_<date>`), jamais supprimée automatiquement ; espace disque contrôlé avant.
- Après restauration : `./deploy.sh <version actuelle>` pour rejouer migrations et droits.

Codes d'échec (sauvegarde)	Codes d'échec (restauration)
`dump_failed`, `verify_failed`, `encrypt_failed`, `upload_failed`, `timeout`, `agent_restarted`	`disabled`, `download_failed`, `checksum_mismatch`, `invalid_key`, `key_missing`, `decrypt_failed`, `restore_failed`, `swap_failed`, `timeout`

Procédure pas à pas : `docs/GUIDE-SAUVEGARDES.md`.

10. Observabilité



- L'API exporte traces, métriques et logs vers `OBSERVABILITY_OTLP_URL` avec `Authorization: Bearer OBSERVABILITY_TOKEN` ; URL vide = aucun envoi.
- `/health` : contrôle de santé Docker et Traefik (toutes les 15 s) ; supervision de l'outbox (alerte si des messages attendent plus de `Outbox:MaxPendingMinutes`).
- Les prestataires en échec sont journalisés en avertissement avec le statut HTTP et le code d'erreur uniquement.
- Agent de sauvegarde : `journalctl -t restauspace-backup` ; alerte d'échec possible via `BACKUP_PING_URL` (ex. healthchecks.io).

11. Exploitation

Commandes utiles

```
# Serveur applicatif (/opt/restauspace/deploy/production)
docker compose ps
docker compose logs -f api
docker compose run --rm migrations          # migrations seules
docker compose up -d --force-recreate api   # après modification de api.env
./deploy.sh <commit>                         # déploiement ou retour arrière

# Serveur de base
sudo restauspace-backup-agent --status
sudo journalctl -t restauspace-backup -n 50 --no-pager
sudo -u postgres psql -d es_restaurant       # peer : toujours via sudo -u postgres
sudo ./tls-ca.sh --renew                     # certificat PostgreSQL (3 ans)
```

Incidents déjà rencontrés

Symptôme	Cause	Correction
`psql` : peer authentication failed	connexion locale sans le bon utilisateur système	`sudo -u postgres psql`, sur le serveur de base
Stripe 401	identifiant `mk_…` collé au lieu de la clé secrète	utiliser `sk_…`
`seed-plans` 404	`/v1` ajouté deux fois	`--api https://api.restauspace.com`
Jenkins : étapes Images à Déploiement sautées	build lancé par le job Multibranch	lancer `release/restauspace-release`
Lien de sauvegarde : `AccessDenied`	politique S3 vide sur l'utilisateur API	ré-importer `ovh-politique-api.json`
`pg_restore: unsupported version (1.16)`	client plus ancien que PostgreSQL 18	`docker run postgres:18-alpine pg_restore …`

Points ouverts avant l'ouverture au public

- Restreindre `ADMIN_ALLOWED_IPS` ; vérifier la 2FA de chaque administrateur.
- Passer la base en `VerifyFull` si ce n'est pas fait ; épingler la clé SSH du VPS dans Jenkins ; activer la clé bridée.
- Tester une restauration complète, puis une fois par trimestre ; désactiver ensuite `RESTORE_ENABLED`.
- Désactiver l'authentification SSH par mot de passe sur le serveur de base.
- File d'emails Identity en mémoire et instance unique d'API (voir `docs/AUDIT-CONTENEURS-PRODUCTION.md`).